Software Testing validation and verification

CSE341, CSE338, CSE337s

Term Project: Designing and Executing Tests for a Banking System

🧩 Overview

Students will test a simplified banking software that includes:

- **Client onboarding and profile management**
- **Deposit, withdrawal, transfer functionalities**
- **Admin approval workflows**
- **State-based transitions for account statuses** They will apply **black-box, white-box, UI, state diagram-based testing**, and simulate **TDD** for selected modules.

Part 1: Provided Artifacts

Students will receive:

- 🖼️ **GUI Screenshots or basic HTML files** for login, account dashboard, transaction pages
- 💻 **Code snippets** in Java/Python for:
  - ClientController
  - TransactionProcessor
  - AccountService
- 🔌 **Integration knowledge** about how modules link (e.g., controller calls service, database updates)
- 🔁 **State Diagram** showing client states: Unverified → Verified → Suspended → Closed

---

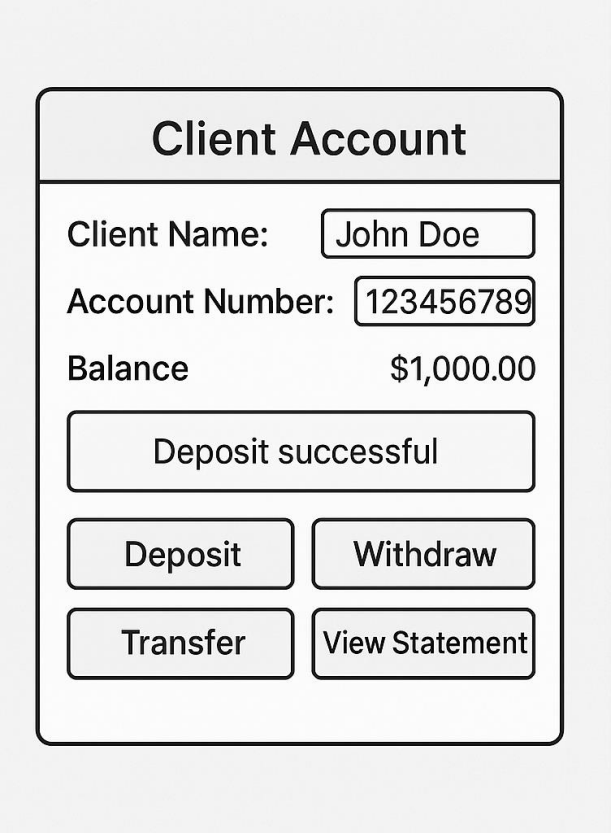
🖼️ **Sample GUI Screen (Text Description)**

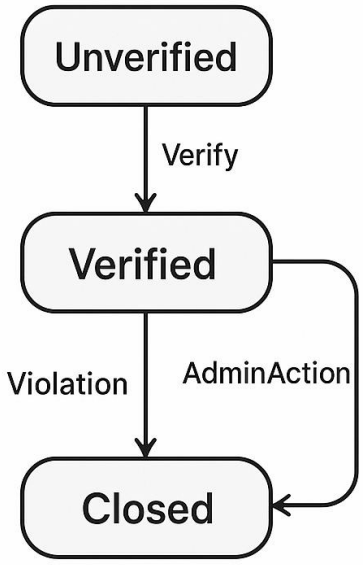
Imagine the Client Dashboard GUI with these elements:

- **Text fields: Client Name, Account Number, Balance**
- **Buttons: Deposit, Withdraw, Transfer, View Statement**
- **Status Label: shows Verified, Suspended, Closed**
- **Notification box: shows success/failure messages**

Students can test UI input validation, button visibility, and state-dependent behavior (e.g. transfer disabled when "Suspended").

🔄 **State Diagram: Account Lifecycle**





Transitions:

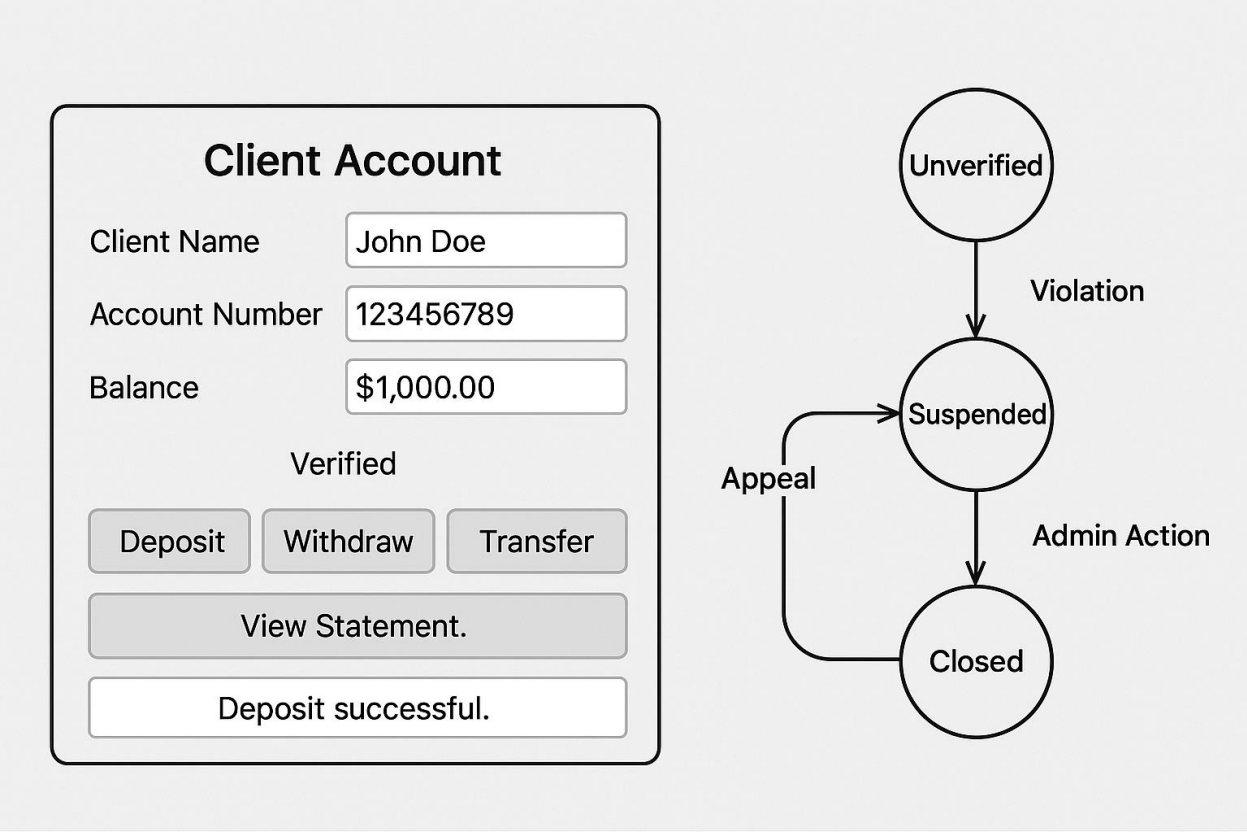
- Verify → Verified

- Violation → Suspended

- AdminAction → Closed

- Appeal → Verified (from Suspended)

Tests can target illegal transitions (e.g. trying to deposit in "Closed" state) and state-dependent feature access.



💻 **Sample Java Code (To Be Tested)**

```
public class Account {
   private double balance;
   private String status; // "Verified", "Suspended", "Closed"

   public Account(double initialBalance, String initialStatus) {
      this.balance = initialBalance;
```

```
        this.status = initialStatus;
    }

    public boolean deposit(double amount) {
        if (status.equals("Closed") || amount <= 0) return false;
        balance += amount;
        return true;
    }

    public boolean withdraw(double amount) {
        if (status.equals("Closed") || status.equals("Suspended")) return false;
        if (amount > balance) return false;
        balance -= amount;
        return true;
    }

    public double getBalance() {
        return balance;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

🧪 **Testing Targets**

✅ Black-Box Testing

Test Case ID	Input	Expected Output	Notes
BB01	deposit(-100)	false	Invalid amount
BB02	withdraw(50)	true if balance ≥ 50	Blocked if suspended
BB03	withdraw(500)	false	Overdraft prevention

Use Equivalence Partitioning:

- **Negative deposit → invalid**
- **Valid deposit → success**
- **Deposit in "Closed" → fail**

⚙️ White-Box Testing

- **Test coverage of deposit() and withdraw() paths.**
- **Create a Control Flow Graph for branching logic.**
- **Aim for 100% branch coverage including:**
  - **Suspended account checks**
  - **Negative amount handling**
  - **Overdraft conditions**

🖱️ UI Testing

Functional checks for:

- **Proper rendering of status label**
- **Buttons disabled based on state**
- **Input validation errors displayed**

Optionally simulate with Selenium (e.g., assertTrue(button.isEnabled())).

🔁 State-Based Testing

State	Allowed Actions	Illegal Actions
Verified	Deposit, Withdraw	-
Suspended	View only	Withdraw, Transfer
Closed	View only	Deposit, Withdraw

Test transitions and expected behavior:

- **Deposit in Closed → fail**
- **Appeal from Suspended → Validates transition back to Verified**

🔗 Integration Testing

Simulate user flow from GUI → Controller → Account methods:

- **Deposit clicked → controller validates → calls Account.deposit()**
- **Mock services where needed**
- **Test response messages on GUI (e.g., "Deposit successful")**

Part 2: Student Deliverables

📁 Section A: Black-Box Testing

- Identify boundary cases for deposits, withdrawals
- Validate behavior against requirements (e.g., transfer limit)
- Create **Equivalence Partitioning** and **Boundary Value Analysis** tables

🧬 Section B: White-Box Testing

- Analyze TransactionProcessor code for:
    - Decision paths
    - Loops and branches

- Create **Control Flow Graphs**
- Write **unit-level test cases** with coverage annotations

🖱️ **Section C: UI Testing**

- Analyze GUI files for:
    - Input validation
    - UX errors
- Create functional tests using tools like Selenium or manual checklists

🕹️ **Section D: State-Based Testing**

- Generate test scenarios from the state diagram
- Example: What happens if a suspended account tries a transaction?
- Design a **state transition matrix** with expected results

🛠️ **Section E: Test-Driven Development (TDD) for New Feature**

- Simulate adding a new feature: "Client Credit Score Check"
- Write test cases before implementation
- Include:
    - Test Plan
    - Expected behavior
    - Stub code + full implementation

📝 **Reporting and Documentation**

Each student (or team) must submit:

| Artifact | Format | Description |

|---|---|---|

| Test Case Documents | Structured Tables | Inputs, expected outputs |

| Code Coverage Report | PDF/Text export | Percentage coverage |

| UI Bug List | Bullet Format | Screenshots and notes |

| TDD Summary | Narrative Report | Thought process + learning points |

| Final Presentation | Slides or Infographic | Summary of all testing types |

Evaluation Criteria

Component	**Weight**
Black & White Box Tests	25%
UI Testing	15%
State-Based Testing	20%
TDD Implementation	25%
Documentation & Creativity	15%